

Grupo

1201560 – Reinaldo Reis

1240160 – Nuno Castro

Metodologia de Teste

Para garantir a validade dos benchmarks, os testes devem ser executados com a flag `–release`. Isto assegura que o compilador Rust aplica todas as otimizações necessárias e remove as verificações de debug, que distorceriam os tempos de execução.

Para relaizar os testes executas o seguinte comando pois assim tiramos a coisas de debug e otimizamos o código

```
cargo run --release
```

Além disso, cada benchmark foi executado 10 vezes consecutivas, sendo calculada a média final, de modo a reduzir variações ocasionais e obter valores mais fiáveis.

Resultados Obtidos

A tabela abaixo apresenta os tempos de execução médios e o ganho de performance (speedup) relativo à implementação sequencial.

Implementação	1000x1000 (ms)	Ganho %	2000x2000 (ms)	Ganho %	5000x5000 (ms)	Ganho %
Sequencial	748.56	0% (Base)	4841.90	0% (Base)	77464.79	0% (Base)
ThreadPool Linha	48.42	93.53%	338.58	93.01%	5908.92	92.37%
ThreadPool Pixel	509.82	31.89%	1956.25	59.60%	25648.64	66.89%
Rayon	46.46	93.79%	367.96	92.40%	11628.80	84.99%

- Cálculo do ganho: $(\text{Tempo sequencial} - \text{Tempo paralelo}) / \text{Tempo sequencial} \times 100$

Análise e Discussão dos Resultados

1. ThreadPool Pixel

A implementação baseada em pixels cria uma tarefa (*task*) independente para cada **ponto** da imagem.

- **O Problema:** Para uma resolução de 1000x1000, são geradas **1.000.000 de tarefas**.
- **Overhead Excessivo:** O custo administrativo de processar cada tarefa supera frequentemente o tempo do próprio cálculo matemático. Este custo inclui:
 1. Alocação de memória para a *closure*.
 2. Envio e gestão da tarefa na fila da *ThreadPool*.
 3. *Context switching* (mudança de contexto) das threads.
 4. Sincronização e envio do resultado pelo canal (*mpsc*).
- **Conclusão:** A granularidade é demasiado fina. O sistema passa mais tempo a gerir tarefas do que a executá-las, resultando num ganho de performance medíocre (30-60%).

2. ThreadPool Linha

A implementação baseada em linhas cria uma tarefa por **linha completa** da imagem.

- **A Vantagem:** Para 1000x1000, são criadas apenas **1.000 tarefas**.
- **Eficiência:** Cada tarefa possui uma carga de trabalho substancial (calcula 1000 pontos). Desta forma, o *overhead* de gestão da tarefa torna-se insignificante face ao tempo de computação útil.
- **Resultado:** O CPU dedica a vasta maioria do tempo aos cálculos efetivos e minimiza o tempo de gestão de threads, atingindo um ganho de **~93%**, o que demonstra uma excelente utilização dos múltiplos núcleos do processador.

3. Comportamento do Rayon

O Rayon utiliza uma estratégia de *work-stealing* (roubo de trabalho), tentando dividir o processamento dinamicamente para equilibrar a carga.

- **Cargas Pequenas/Médias:** O desempenho é excelente, equiparando-se à *ThreadPool* manual, provando que o seu *overhead* de abstração é baixo.
- **Cargas Elevadas (5000x5000):** Neste cenário, o Rayon foi aproximadamente 2x mais lento que a *ThreadPool* manual (11s vs 5.9s).
 - **Análise:** A *ThreadPool* manual (Linha) é determinística e estática: aloca exatamente 1 linha por tarefa. O Rayon, ao tentar ser "inteligente" e dividir o trabalho recursivamente, acabou por introduzir um *overhead* desnecessário numa carga de trabalho que é altamente regular e previsível. A simplicidade da divisão por linhas revelou-se mais eficaz para este conjunto de dados massivo.

Conclusão

Em suma, este estudo valida que o paralelismo é indispensável para a escalabilidade de problemas computacionalmente intensivos, permitindo reduzir tempos de execução de minutos para segundos, mas sublinha que a estratégia de decomposição é tão crítica quanto a própria paralelização. A ineficiência observada na abordagem píxel a píxel, em contraste com o desempenho ótimo da divisão por linhas, demonstra que o controlo da granularidade e a minimização do overhead de gestão de tarefas são os fatores determinantes para a eficiência, provando ainda que, em cargas de trabalho massivas e previsíveis, uma gestão manual de threads (ThreadPool) pode superar ligeiramente a conveniência de abstrações automáticas como o Rayon.